QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **4**

1. Scope & Requirements Traceability 4
2. Risk Assessment & Mitigation Strategy 5
3. Test Environment & Execution Strategy 8
4. CI/CD Release Thresholds & Automation Gates 11
5. Test Case Specifications (Drafts) 13
6. Test Strategy & Plan Sign-Off 16

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team **SellerMate***
Team Repo URL	https://github.com/Chin123-hash/UMHackathon
Project Board URL	https://trello.com/invite/b/69eb229659a2d037f7f932a3/ATTI0047ae3cc2c932baa9bdddd6a0c680024873469B/my-trello-board
Live Deployment URL (optional for preliminary round)	*[Insert link to the live hosted application]*

Objective:

*The primary objective of quality assurance testing for **SellerMate** is to ensure that the AI-powered social commerce assistant can reliably process and manage end-to-end chat-based workflows, including customer inquiry handling, order extraction, short forms understanding, inventory management, and shipping coordination.*

The system must accurately interpret unstructured and informal chat inputs (e.g., slang, abbreviations, mixed-language messages), maintain contextual conversation memory, and execute multi-step workflows such as retrieving data from databases, validating stock availability, and recommending suitable courier options. In addition, the testing aims to verify:

- *The correctness and consistency of AI-generated responses*
- *The accuracy of order details extracted from chat conversations*
- *Real-time synchronization between chat inputs, inventory data, and order records*
- *The system's ability to handle high message volumes (e.g., bulk ordering scenarios)*
- *Robust handling of edge cases, including ambiguous inputs, missing data, and insufficient stock*

Ultimately, the goal is to ensure that SellerMate operates reliably, reduces manual workload for sellers, minimizes operational errors, and delivers a fast, accurate, and seamless experience for both sellers and buyers in social commerce environments.

© UMHackathon 2026

Email: umhackathon@um.edu.my

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section ensures that all testing activities are aligned with SellerMate's functional requirements through a **Requirement Traceability Matrix (RTM)**. It verifies that each core feature is properly tested to prevent defects, ensure reliability, and avoid unintended feature expansion.*

1.1 In-Scope Core Features

- **Chat Understanding & Processing:**
  - Interpretation of customer messages (including slang, short forms, mixed-language inputs)
  - Intent detection (e.g., inquiry, order placement, product question)
  - Context retention across conversations (partial stateful memory)
- **Order Extraction & Management:**
  - Extraction of structured order details from unstructured chat
  - Order summarization and confirmation
- **Inventory Validation & Management:**
  - Real-time stock checking before order confirmation
  - Detection and handling of insufficient stock scenarios
  - Synchronization between chat inputs and inventory database
- **Workflow Orchestration (End-to-End Automation):**
  - Multi-step workflow: chat → intent analysis → data retrieval → decision-making
  - Automated order validation and confirmation process
  - Integration with backend systems (inventory, order database)
- **AI Reliability & Fallback Mechanism:**
  - Automatic fallback from primary AI (Z.AI) to secondary AI (Groq)
  - Seamless handling of API failure without disrupting user experience
- **AI Response Generation:**
  - Context-aware replies instead of fixed templates
  - Clarification prompts for ambiguous or incomplete inputs
  - Consistency and accuracy of responses
- **Daily Sales Briefing Generation (English):**
  - Generate concise business summaries
  - Highlight top-performing products
  - Provide actionable stock-related recommendations
- **AI Business Analytics:**
  - **Identification of Bottlenecks**
  - **Actionable Business Recommendations**
  - **Sales Anomaly Detection & Explanation:**
    - Identify unusual sales patterns (spikes/drops)
    - Generate short explanations for anomalies (e.g., trends, stock issues)

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Predictive Insights & Stock Recommendations:**
  - Suggest restocking strategies
  - Provide forward-looking insights based on sales patterns
  - Recommend actions to improve inventory health
- **Courier Selection & Shipping Coordination:**
  - Comparison of courier options (cost, delivery time, availability)
  - Recommendation of optimal courier
  - Generation of shipping details and tracking information

1.2 Out-of-Scope

- Promotions & Marketing Features
- User Profile Management: Edit profile info, and account settings
- UI/UX Customization
- Advanced Enterprise Analytics (Beyond AI Prompts): Using complex dashboard such as BI tools and generating financial reporting

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Inventory database fails to update after order confirmation (overselling)	*4*	*5*	*20 (Critical)*	*Implement transaction locking & real-time sync between order and inventory system*	*Simulate concurrent orders (stress test) and verify stock consistency*
AI server downtime / unavailable API	*3*	*5*	*15 (High)*	*Add fallback mechanism (basic rule-based replies) + retry logic + monitoring alerts*	*Simulate API failure and verify fallback responses are triggered*
AI fails to extract correct order details from chat (wrong quantity/product)	*4*	*5*	*20 (Critical)*	*Add confirmation step before finalizing order + validation rules*	*Use varied chat inputs (slang, mixed language) and verify extracted results*
AI misinterprets customer intent (e.g., inquiry vs	*3*	*4*	*12 (High)*	*Improve prompt design + intent classification layer +*	*Test ambiguous messages and evaluate AI response*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
order)				*clarification prompts*	*accuracy*
AI fails to generate restock recommendations (missed low stock alert)	*3*	*3*	*9 (Medium)*	*Set threshold-based triggers independent of AI + scheduled checks*	*Simulate low stock scenarios and verify alerts are triggered*
AI generates inaccurate business insights (wrong analysis or misleading advice)	*3*	*4*	*12 (High)*	*Constrain prompts + validate input data + limit output format*	*Feed incorrect/edge data and verify output reliability*
Database latency or slow response affecting real-time workflow	*3*	*4*	*12 (High)*	*Optimize queries + caching + timeout handling*	*Perform load testing and measure response time*
Failure in multi-step workflow (chat → DB → decision → response breaks mid-process)	*3*	*5*	*15 (High)*	*Implement workflow checkpoints + rollback/retry mechanism*	*Force failure at each step and observe system recovery*
Courier API failure or incorrect shipping recommendation	*2*	*4*	*8 (Medium)*	*Provide default courier fallback + error handling*	*Simulate API failure and verify fallback logic*
Data inconsistency between chat, orders, and inventory systems	*3*	*5*	*15 (High)*	*Use centralized database + consistency checks*	*Compare data across modules after operations*
High message volume causes system lag or crash	*4*	*5*	*20 (Critical)*	*Implement load balancing + queue system (e.g., message buffering)*	*Stress test with bulk messages ("plus 1" flood simulation)*
AI hallucination (generates information not based on actual data)	*3*	*4*	*12 (High)*	*Restrict AI to structured data + grounding with database*	*Test with incomplete data and verify responses*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Primary AI (Z.AI) failure and fallback (Groq) not triggered	*3*	*5*	*15 (High)*	*Implement automatic fallback logic with retry mechanism*	*Simulate Z.AI failure and verify Groq response*
Inconsistent output between Z.AI and Groq	*4*	*4*	*16 (Critical)*	*Standardize prompts and enforce output format validation*	*Compare outputs from both APIs*
AI analytics produces misleading business insights	*3*	*4*	*12 (High)*	*Validate AI outputs against actual data*	*Feed controlled datasets and verify accuracy*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
  - *Scope:*
    - *Chat parsing & intent detection logic*
    - *Order extraction (product, quantity, variation)*
    - *Inventory validation logic*
    - *AI prompt formatting (input/output structure validation)*
    - *Business logic (e.g., stock threshold checks, courier selection rules)*
  - *Execution:*
    - *Tests are written using frameworks such as Jest (Node.js) or PyTest (Python)*
    - *Executed during development and automatically triggered in CI pipeline upon code push*
  - *Isolation:*
    - *External dependencies (AI API, database, courier API) are mocked*
    - *Focus only on internal logic correctness (e.g., extraction accuracy, validation rules)*
  - *Pass Condition:*
    - *Handles happy paths, negative cases, and edge cases*
    - *Returns correct structured outputs*
    - *Proper error handling for invalid or ambiguous inputs*

- **Integration Test**
  - *Scope:*
    - *Chat input → AI processing → database → response generation*
    - *Inventory system ↔ order management*
    - *AI outputs ↔ backend validation*
  - *Execution:*
    - *Conducted after unit testing is completed*
    - *Performed when integrating multiple modules (e.g., AI + database + workflow engine)*
  - *Workflow:*
    - *Uses real API calls with the following workflow: Customer message → AI extraction → DB check → decision → reply*
  - *Pass Condition:*
    - *All components communicate correctly*
    - *Data flows accurately across modules*
    - *End-to-end workflow produces expected results (e.g., correct order + stock deduction)*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Test Environment (CI/CD Practice):**
    - *Local Testing:*
        - *Manual testing on localhost*
        - *Developers test chat scenarios and AI responses directly*
    - *Staging/CI Environment :*
        - *Use GitHub Actions (or similar CI tools)*
        - *Automatically runs unit + selected integration tests on every push*
    - *CI/CD automated pipeline:*
        - *On each commit → run unit tests*
        - *On Pull Request → run integration tests + workflow tests*
        - *On merge to main → run full regression suite*
    - *Environment Setup: System integrates multiple AI APIs (Z.AI and Groq) with fallback routing logic tested in staging environment.*

- **Regression Testing & Pass/Fail Rules:**
    - *Execution Phase:*
        - *Triggered whenever new features or fixes are merged*
        - *Covers all in-scope core features: Chat processing, Order extraction, Inventory validation, and AI insights generation*
    - *Pass/Fail Condition:*
        - *A test case passes only when actual output matches expected output*
        - *AI-related tests allow controlled tolerance (e.g., correct intent + correct structure)*
        - *Any mismatch is logged in Defect Log*
    - *Continuation Rule:*
        - *Critical modules such as order extraction and inventory validation must pass before proceeding*
        - *If critical tests fail → pipeline is blocked*

- **Test Data Strategy:**
    - *Manual Test Data: Create mock scenarios including:*
        - *Normal orders ("I want 2 size M shirts")*
        - *Slang/mixed language ("nak 1 baju M")*
        - *Invalid/ambiguous inputs*
    - *Automated Test Data: Use seed scripts to generate:*
        - *Product inventory (normal + low stock + out of stock)*
        - *Order records*
        - *Sales data (for AI insights & anomaly detection)*
    - *AI Testing Data: Predefined datasets for daily sales summary, anomaly scenarios, and stock health conditions*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Passing Rate Threshold:**
    - *Minimum overall pass rate: 85%*
    - *Critical features (must be 100% pass): Order extraction accuracy, Inventory validation, and Workflow execution (chat → order → confirmation)*
    - *AI-related tolerance:*
        - *Output must be factually correct + contextually relevant*
        - *Minor wording differences are acceptable*
        - *Incorrect facts or wrong decisions = FAIL*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***0 Errors***	***Passed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***79 Error***	***Failed***
Test Coverage	***Minimum 80%***	***90%***	***Passed***
AI Output Structure Validation	***Minimum 90% correct format (JSON/order extraction)***	***100%***	***Passed***
Workflow Execution	***Minimum 90% success (chat -> AI -> DB)***	***100%***	***Passed***
AI Fallback Success Rate	***Minimum 95%***	***100%***	***Passed***
AI Output Consistency	***Minimum 80%***	***85%***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 75% of prompts correct***	***80%***	***Passed***
Critical Bugs	***Zero P0/01 Bugs***	***0***	***Passed***
API Performance	***Response Time < 800ms***	***50000ms***	***Failed***
Security	***API keys are not exposed***	***Clean***	***Passed***
End-to-End Workflow Success Rate	***Minimum 95% (chat -> order -> confirmation)***	***100%***	***Passed***
Database Consistency	***100% (no stock mismatch)***	***100%***	***Passed***
System Availability	***Minimum 99% fallback ready***	***100%***	***Passed***
AI Fallback Reliability	***Minimum 99% successful fallback execution***	***100%***	***Passed***
AI Insight Accuracy	***Minimum 75% correct recommendation***	***80%***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	***Happy Case (Entire Flow):*** *Chat → Order → Inventory → Confirm Order*	*Verify that SellerMate can process a complete customer interaction from chat input to order confirmation, including inventory validation and shipping coordination.*	1. *Customer sends message: "Hi, I want 2 black shirts size M"* 2. *AI interprets and extracts order details* 3. *AI checks inventory availability* 4. *AI gets a shipping address from the customer to select the best courier based on address* 5. *AI chose the best courier options.* 6. *AI calculates the total price including product price and shipping fees.* 7. *Customer confirms order.* 8. *AI sends order confirmation to the seller and database*	*Order is correctly extracted, stock is validated, and order is successfully confirmed. Shipping details are generated and sent within 5 seconds.*	*Order successfully extracted with correct quantity and variation. Stock deducted correctly. Courier assigned and tracking info generated within 1.5 seconds.* *Status: Passed*
TC-02	***Specific Case (Negative):*** *Insufficient Stock Handling*	*Verify that the system correctly handles cases where customer order exceeds available inventory.*	1. *Inventory has 1 units of product* 2. *Customer sends message: "I want 10 units"* 3. *AI extracts order and triggers stock validation*	*The AI blocks order confirmation and responds with a clear message: "Only 1 unit available" and suggests*	*The AI blocks the order with a response message "I'm sorry, sis, for the Blouse Chiffon Raya Exclusive in size M, we only have 1 unit left in*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			4. *AI evaluates stock availability*	*adjustment. No stock deduction occurs.*	*stock. Would you like to take that 1 unit?". If the customer persists to buy more than 1 unit, the AI will response with "I am very sorry, sis, the stock for the Blouse Chiffon Raya Exclusive in size M really only has 1 unit left right now, so it's not enough for the 3 units [you requested]. Do you want to take the 1 remaining unit? If you agree, I can proceed with just 1 unit for you." No stock deduction.* *Status: Passed*
TC-03	***NFR (Performance) :*** *Workflow Execution Stability*	*Verify stability of end-to-end workflow under concurrent operations (chat → AI → DB → response).*	1. *Simulate 100 concurrent users placing orders* 2. *Each triggers full workflow (chat → extraction → stock check → confirmation)* 3. *Monitor system success rate and errors*	*Minimum 95% successful workflow completion with no data inconsistency and response time less than 3 seconds.*	*94% workflows completed successfully. Minor delays observed. 1 case of delayed stock update detected.* *Status: Failed*
TC-04	***NFR (Performance) :*** *AI Response & Order Processing*	*Verify system performance when multiple users send order-related*	1. *Simulate 100 concurrent users sending order messages (e.g., "I want 2 shirts*	*Minimum 95% of requests processed successfully with average response*	*93% of requests processed successfully. Average response time = 2.6 seconds.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
	Time	*messages concurrently, focusing on AI response time and workflow execution speed.*	*size M")* *2. Each message triggers full workflow (chat → AI extraction → inventory check → response)* *3. Measure response time and success rate*	*time less than 2 seconds and no system crash or timeout*	*No crash observed, but slight delays under load* *Status: Failed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Hi! Saya nak beli 2 helai Baju Kurung Moden size M. Hantar ke No 5, Jalan Bangi, 43650 Selangor. Nama saya Farhana.	- Output in Malay - Check inventory - Extract the state "Selangor" and calculate the shipping rate - Calculate the total price without error	Saya cek stok Baju Kurung Moden Sulam Bunga – RachelFashion size M, dan ada 4 unit tersedia. Jadi, saya boleh proceed dengan pesanan 2 unit yang Farhana nak. Harga untuk Baju Kurung Moden Sulam Bunga – RachelFashion size M adalah RM89.9 per unit. Sebab Farhana beli 2 unit, jadi total harga adalah RM89.9 x 2 = RM179.8. Untuk penghantaran, saya cek harga penghantaran ke Selangor, dan harga paling murah adalah RM8.00. Ini harga paling murah kami dah check untuk sis. Jadi, total yang perlu Farhana bayar adalah RM179.8 (baju) + RM8.00 (penghantaran) = RM187.8. Betul ke, Farhana?	Pass
AI-02	Hello, do you have the Chiffon Blouse in stock? I need 1 unit sent to Penang. Address: 12 Gurney Drive, 10250 George Town.	- Check the stock and the product - Check the shipping fee - Ask for further details	We have the Blouse Chiffon Raya Exclusive – RachelFashion in stock. However, I need to check the available sizes. We have the following sizes available: S, M, and L. For the S size, we have 2 units available (p-002-S). For the M size, we have 1 unit available (p-002-M). For the L size, we have 2 units available (p-002-L). Which size would you like to purchase? Also,	Pass

© UMHackathon 2026

			may I have your name, please? (And don't worry, I've taken note of your address in Penang. I'll calculate the shipping cost once you've confirmed your order.)	
AI-03	Spike in “Tudung Bawal Premium Sulam – 10 Warna”. Explain possible reasons.	- Output based on seller preference language - Explains the anomaly - Reasons must be logic	The sales of Tudung Bawal Premium Sulam – 10 Warna have increased by 200%. This significant surge can be attributed to the growing demand for high-quality, fashionable, and versatile headscarves. The premium product, available in 10 different colors, likely appeals to customers seeking stylish and comfortable options for everyday wear. The substantial increase in sales may also be driven by effective marketing strategies, positive word-of-mouth, and the brand's reputation for delivering excellent products.	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	~1500 tokens (approx. 800–1000 words)
Input used while testing	5000+ words of mixed sales data and chat logs (≈3x limit)
Expected Behavior	- System should truncate or chunk input - Or reject with error message - No crash or timeout
Actual Behavior	Input partially processed, but output became incomplete and inconsistent
Status	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

Field	Details
Test Prompt	“Nak beli baju tu... hantar kat sana.”
Expected Behaviour	- AI should detect ambiguity - Ask for clarification (product, size, quantity) - Avoid making assumptions
Actual Behaviour	Baik, sis! Tapi saya perlukan sedikit maklumat lagi: 1. Size berapa sis nak? S, M, atau L? 2. Boleh bagi nama sis untuk order ini? Alamat di Penang tu saya dah rekod ya.
Status	Pass

6.4. Hallucinating Handling

Our system will detect the hallucination by cross-checking AI outputs with inventory database and sales data input, as well as reject output that contains non-existent products and incorrect stock values.
Besides, the mitigation strategy of our system is using structured prompts with strict constraints. Outputs from both AI services are validated against system data to ensure consistency across fallback scenarios.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my